

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include "commonType.h"
4 #include "initAndTerm.h"
5
6 int createBFSGraph(graphType *g, int totalCnt) {
7     int v;
8
9     if (g->n != 0) {
10         releaseNearListHead(g, g->n);
11         releaseBFSGraph(g);
12     }
13     g->n = 0;
14     g->nearListHead = (graphNode**)malloc(sizeof(graphNode*)*totalCnt);
15     g->visited = (int*)malloc(sizeof(int)*totalCnt);
16     for (v = 0; v < totalCnt; v++) {
17         g->nearListHead[v] = NULL;
18         g->visited[v] = FALSE;
19     }
20     return 0;
21 }
22 int releaseBFSGraph(graphType *g) {
23     free(g->nearListHead);
24     free(g->visited);
25     return 0;
26 }
27 int releaseNearListHead(graphType *g, int totalCnt){
28     int v;
29     graphNode *curP=NULL, *nextP=NULL;
30     for(v = 0;v < totalCnt; v++){
31         nextP = g->nearListHead[v];
32         while(nextP){
33             curP = nextP;
34             nextP = curP->link;
35             free(curP);
36         }
37     }
38 }
39 int createVertexList(int **vList, int totalCnt) {
40     *vList = (int*)malloc(sizeof(int)*totalCnt);
41     return 0;
42 }
43 int releaseVertexList(int **vList) {
44     free(*vList);
45     return 0;
46 }
47 LQType* createLQ(){
48     LQType *LQ;
49     LQ = (LQType *)malloc(sizeof(LQType));
50     LQ->front = NULL;
51     LQ->rear = NULL;
52     return LQ;
53 }
54 int releaseLQ(LQType **LQ){
55     free(*LQ);
56     return 0;
57 }
58
```